

clusterAI

Ciencia de Datos en Ingeniería Industrial

UTN BA

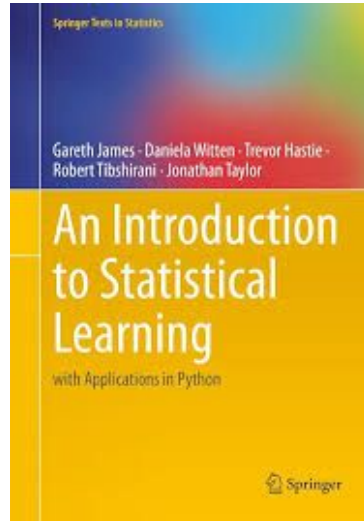
curso I5571

clase_05: Regresion

agenda clase05: aprendizaje supervisado

- Regresion
- Errors metrics para evaluación del modelo
- Regresión Lineal Multivariada
- Ridge regression
- Lasso regression
- Support Vector Regression
- Code

Lectura sugerida

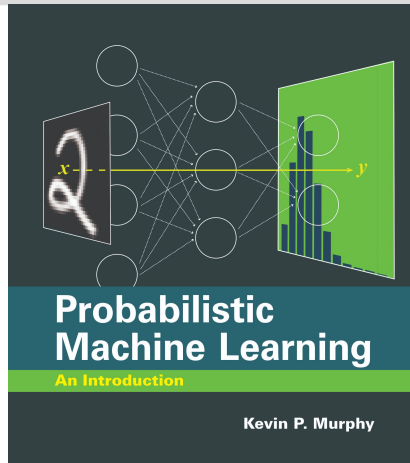


Introduction to Statistical Learning (with applications in Python)

<https://www.statlearning.com/>

https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html

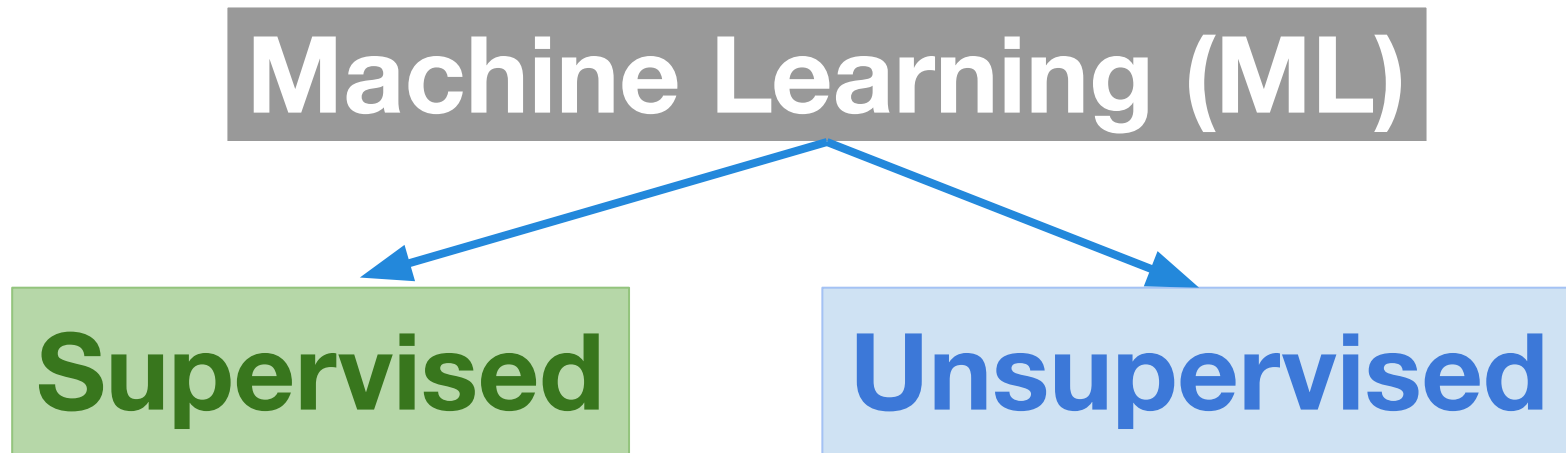
- Capítulo 3 -> Página 69 a 86 (Regresión lineal)



Probabilistic Machine Learning: An Introduction

<https://probml.github.io/pml-book/book1.html>

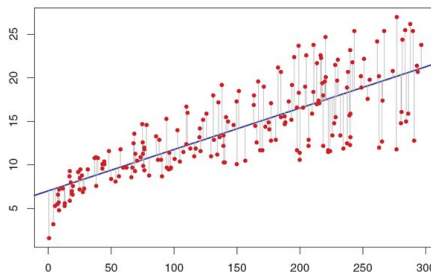
- Capítulo 1 -> Introducción
 - 1.1: What is machine learning?
 - 1.2: Supervised Learning
- Capítulo 11 -> Linear Regression
- Capítulo 17 -> Kernel Methods
 - 17.3: Support vector machines (SVMs)



Particularmente en este curso vamos a poner foco en el aprendizaje **supervisado** y el aprendizaje **no-supervisado**. Estos dos enfoques suelen ser los más populares y prácticos para la mayoría de los problemas.

Métodos de aprendizaje supervisado

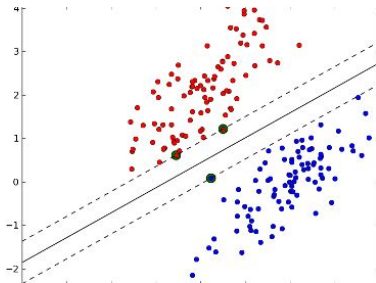
regression



Y es continua

$$y \subseteq \mathbb{R}$$

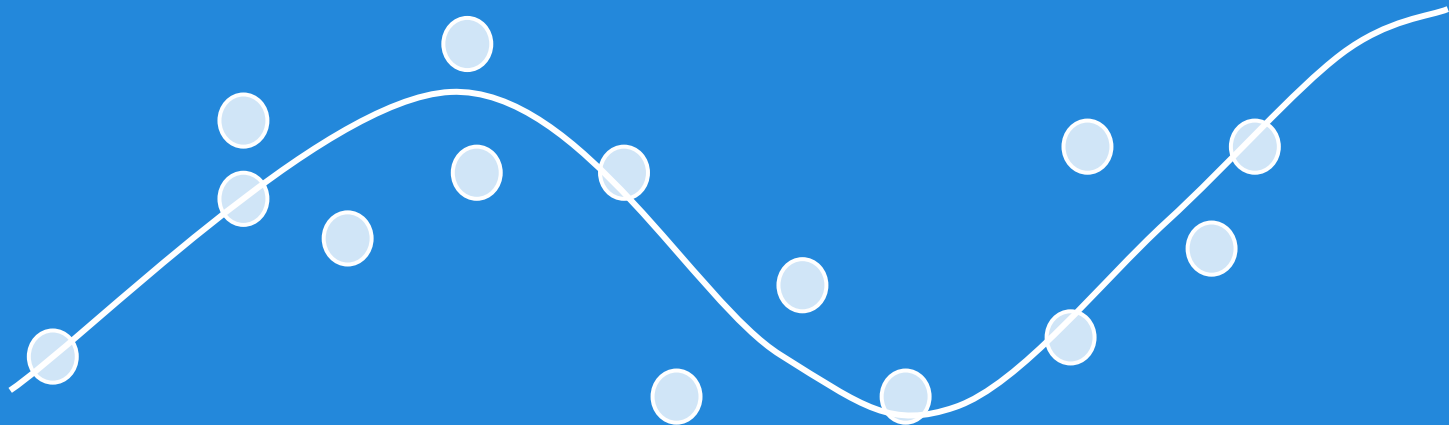
classification



Y es categorica

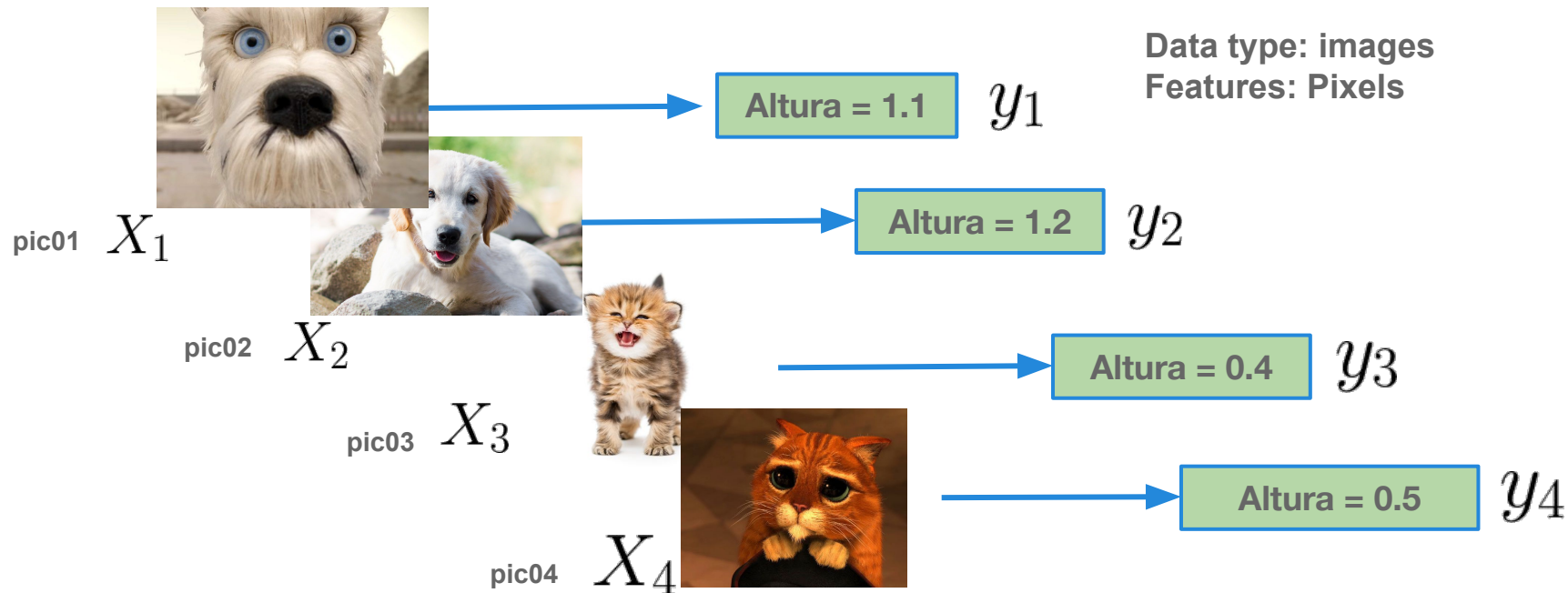
$$y \in \{-1, 1\}$$

Existen dos enfoques importantes en el aprendizaje supervisado: clasificación y regresión. Cuando las etiquetas toman valores categóricos hablamos de clasificación. Cuando las etiquetas toman valores continuos hablamos de regresión.



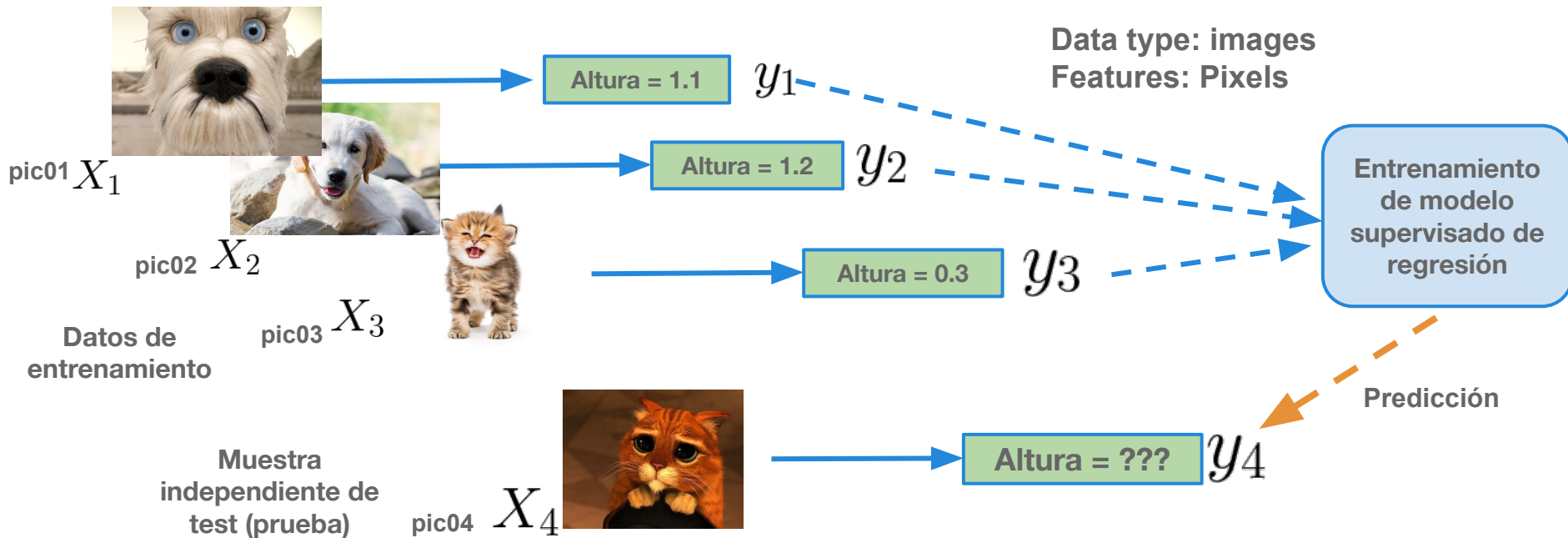
Aprendizaje supervisado: regresión

supervised learning: regression



Cada muestra está asociada a una etiqueta continua aplicada por un humano.

supervised learning: regression

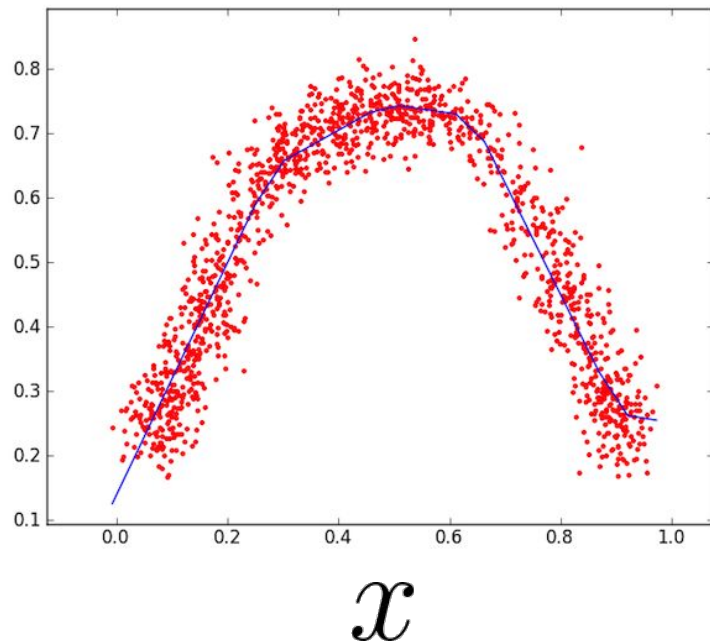


Cada muestra está asociada a una etiqueta continua. No hay clases.

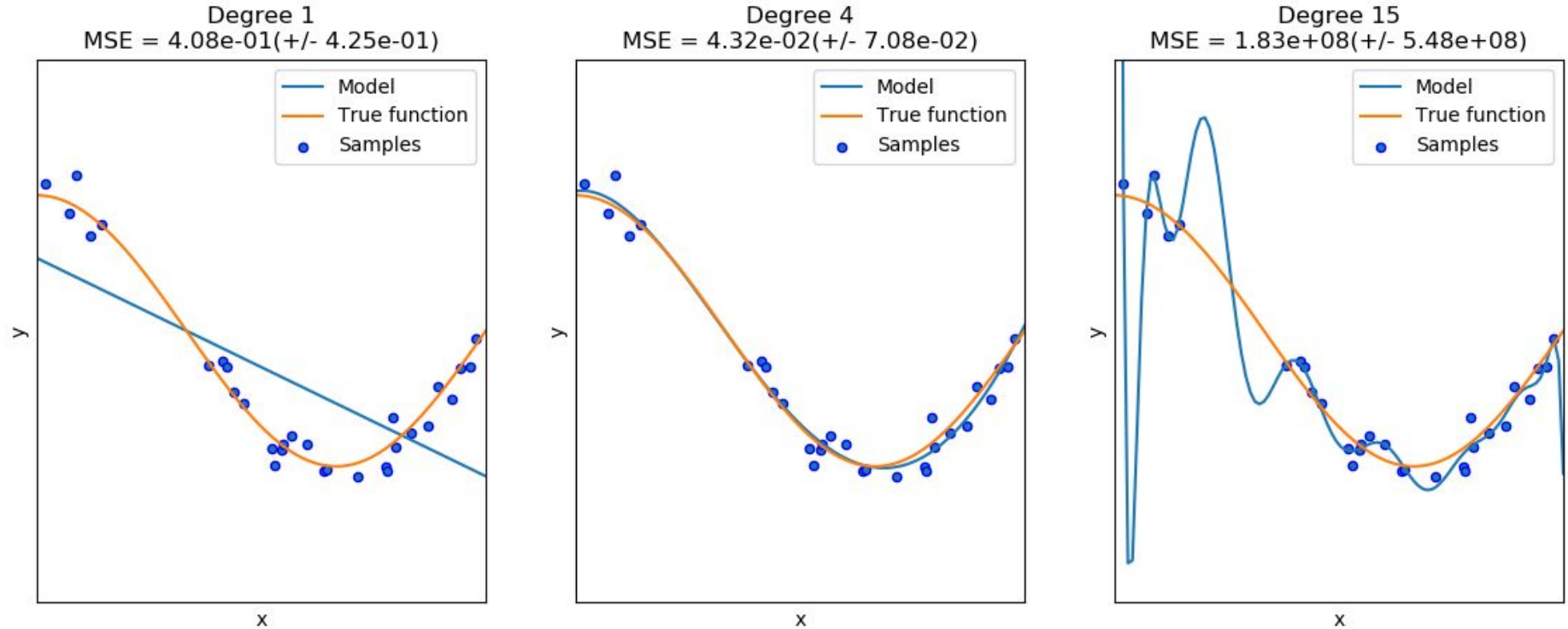
Funciones regresoras

- Es aquella función $F(x)$ que a partir de ciertas “features”/variables de una muestra, estima un valor continuo (real) de la variable dependiente “Y” de esa misma muestra.
- La función regresora se ajustará a los datos de manera tal de minimizar el error entre la predicción “ $\hat{y}$ ” y el valor de la etiqueta “ y ”.

$$\hat{f}_w(x)$$



Underfitting vs Overfitting

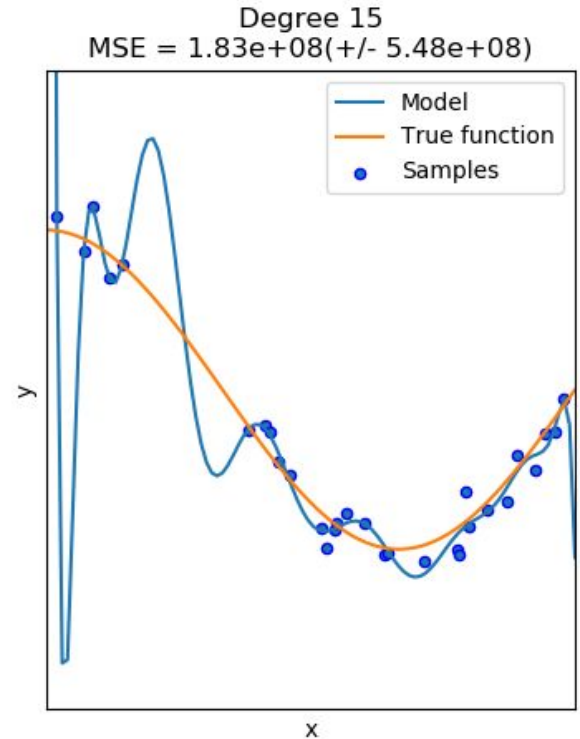


Dado una muestra de datos y suponiendo que conocemos la función que los genera (estocásticamente) ¿cuál modelo es mejor? ¿uno complejo o uno sencillo?

Overfitting

Un modelo de aprendizaje supervisado sea regresión o clasificación puede tener dos problemas: overfitting (sobre-ajuste) o underfitting (sub-ajuste).

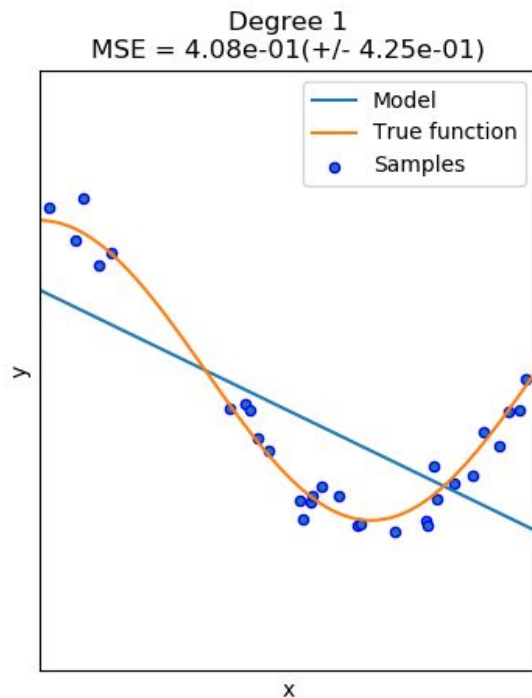
Overfitting: ajustarse demasiado a los datos de entrenamiento para reducir el error de entrenamiento aún cuando suponemos que estos datos no son 100% representativos de la población de donde fueron obtenidos. Es decir, es imposible tener suficiente cantidad de datos como para asegurar que minimizando el error en entrenamiento minimizamos el error de predicción para futuras muestras no vistas. Una forma de observar el overfitting es cuando en train tenemos poco error y en muestras de test independientes tenemos alto error. Sin embargo existen casos donde el error en train y test puede ser muy bajo y aún así realizar overfitting. **El overfitting suele estar asociado a modelos más complejos que se ajustan muy bien a los datos de train aunque generalizan mal para datos futuros.**



Underfitting

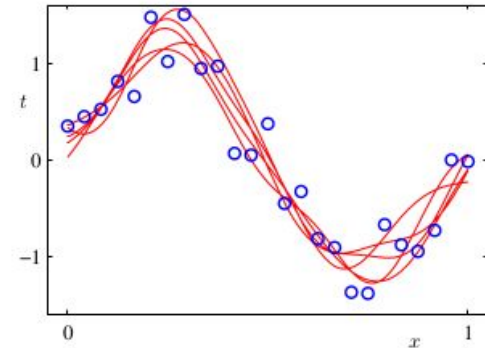
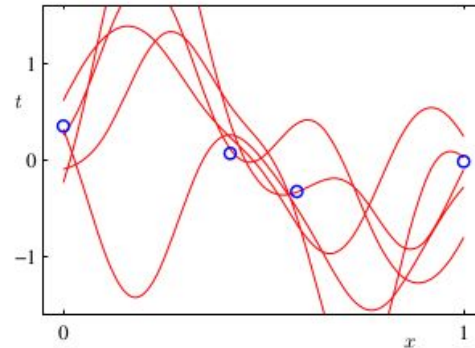
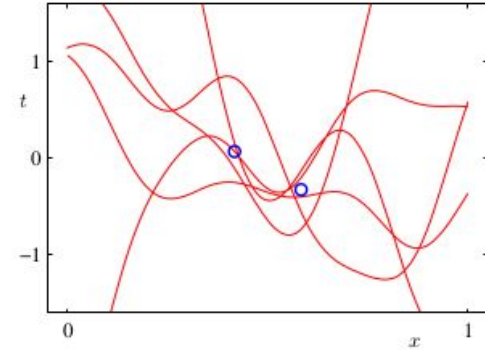
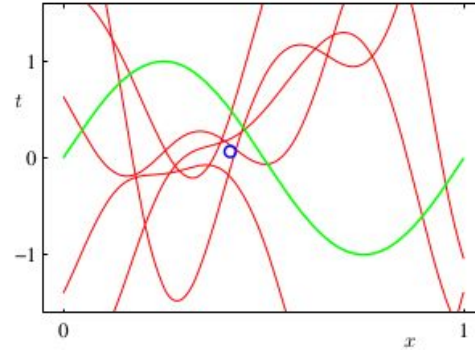
Un modelo de aprendizaje supervisado sea regresión o clasificación puede tener dos problemas: overfitting (sobre-ajuste) o underfitting (sub-ajuste).

Underfitting: ajustarse poco a los datos de entrenamiento para evitar el sobre-ajuste (overfitting) puede generar un error (Bias) que podría haber sido disminuido y no se hizo. Sub-ajustar un modelo a los datos generará menos variación en resultados futuros aunque un resultado sub-óptimo. **El underfitting suele estar asociado a modelos muy sencillos que no logran ajustarse a la complejidad de los datos (ej. Modelos lineales).**

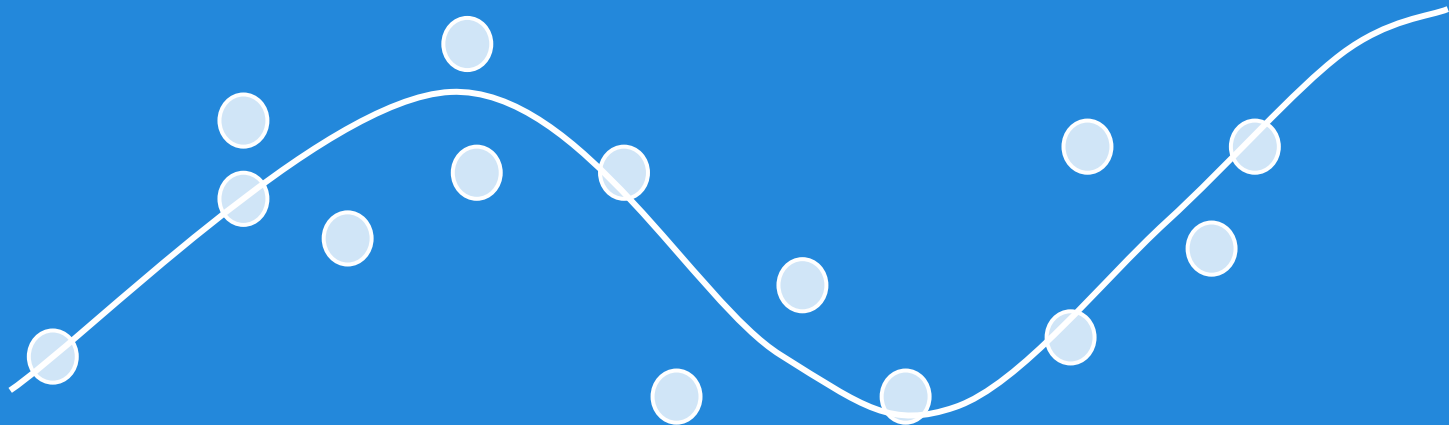


Regression functions

- La cantidad “ n ” de muestras (samples), determinará el ajuste que realice la función (recordar la curva de aprendizaje).
- Más muestras de entrenamiento (si son representativas de la distribución poblacional) implica comprender mejor la verdadera distribución de mis datos y lograr predicciones en datos futuros con menor error.



*Bishop, Pattern Recognition

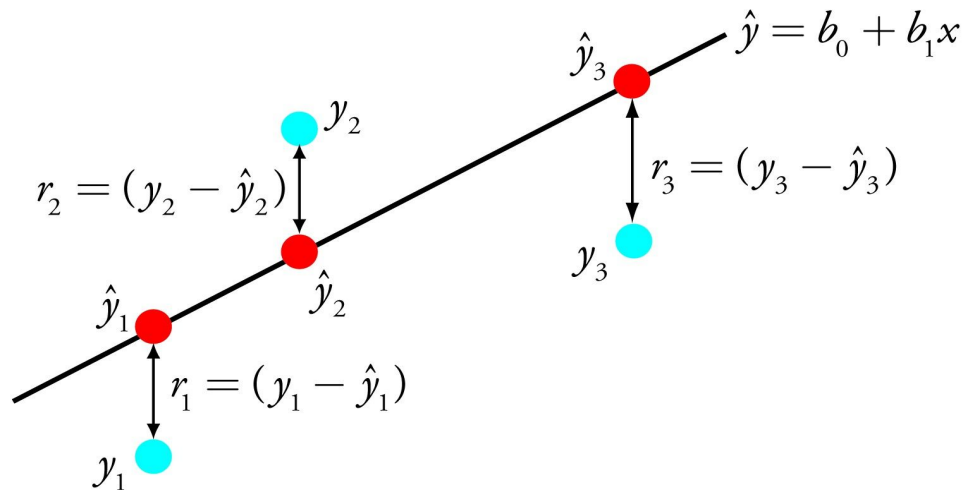


Regresión: medidas de desempeño/performance

Medidas de Error en regresión: como se cuan bueno es mi modelo?

$$e_i = y_i - \hat{y}_i$$

$$\text{RSS} = e_1^2 + e_2^2 + \dots + e_n^2$$



Cada dif. Entre predicción y valor real se denomina residuo (e). Valores más chicos son mejores ya que aproximamos mejor. La sumatoria de los residuos al cuadrado se llama Suma de Residuos al Cuadrado (RSS = Residual Sum Squares)

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}}$$

$$\text{TSS} = \sum_{i=0}^{i=n} (y_i - \bar{y})^2 \quad \text{RSS} = e_1^2 + e_2^2 + \dots + e_n^2$$

El TSS (Total Sum Squares) mide la varianza total de las etiquetas “**y**”. El R^2 explica la proporción de la varianza de “**y**” que explica el modelo de regresión. El R^2 toma valores entre -inf y 1 y es independiente de la escala de “**y**”.

Medidas de Error en regresión: como se cuan bueno es mi modelo?

$$MSE = \frac{\sum (\hat{y}_t - y_t)^2}{n}$$

$$RMSE = \sqrt{\frac{\sum (\hat{y}_t - y_t)^2}{n}}$$

$$MAE = \frac{|\sum (\hat{y}_t - y_t)|}{n}$$

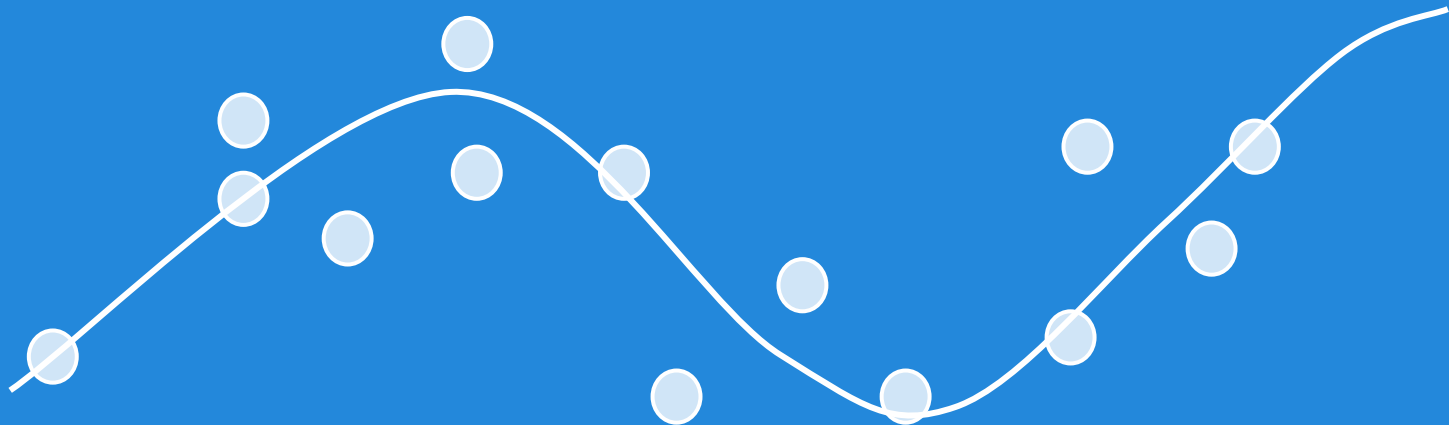
Otras medidas de error en regresión:

1. MSE: Mean Squared Error (error cuadrático medio)
2. RMSE: Root Mean Squared Error (Raiz cuadrada del error cuadrático medio)
3. MAE: Mean Average Error (Media del error)

Por otro lado se observa que:

- MSE es sensible a predicciones **muy** malas (residuos grandes). Puede ser problemático en datos ruidosos.
- El RMSE y MAE computa el error en la misma magnitud de la variable dependiente.
- El MAE es mejor cuando no queremos penalizar fuerte errores grandes (ej. outliers).

Queremos encontrar una $f(x)$ que minimice el error E.



Regresión: regresion lineal multivariada

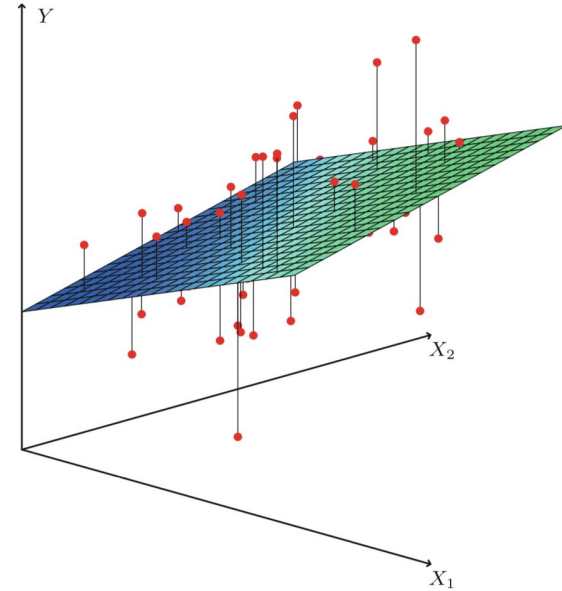
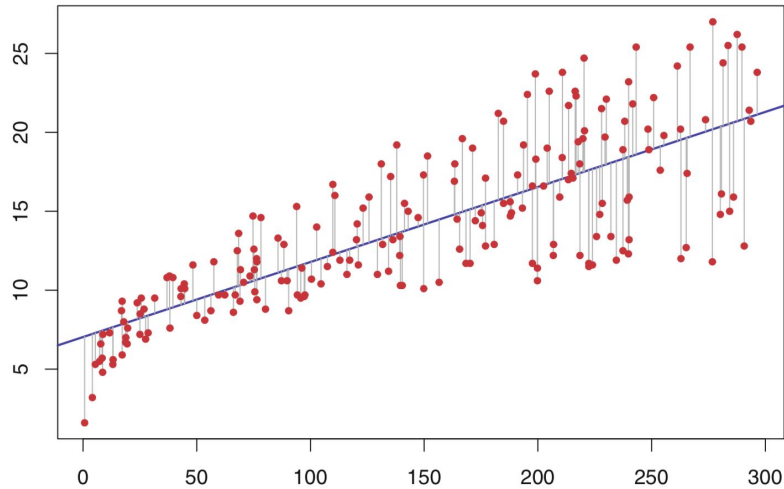
Regresión lineal

$$f(x) = w^T x = y$$

Función de regresión lineal

Existen muchos tipos de funciones de regresión. La familia de funciones más conocida es la de las funciones lineales. Estas funciones son hiper-planos caracterizados por parámetros **w** (vector $w=[w_1 \dots w_d]$) que determinarán los valores que tomará la variable dependiente “**y**” partiendo de los vectores “**x**” que viven en el hiper-espacio de dimensión “**d**”.

Regresión Lineal



Por medio de una función lineal caracterizada por parámetros $\mathbf{w}$ ($d = 1$ izq., $d = 2$ der.) se buscará minimizar la suma de los residuos al cuadrado. Esta función lineal generará una superficie de regresión en dimensión $\mathbf{d}$.

Regresion Lineal

$$\hat{y} = f(x, w)$$

$$\hat{y}(x, w) = w_0 + w_1x_1 + w_2x_2 + \dots + w_px_p$$

Es una función lineal que se construye calculando parámetros “**w**” asociados a cada dimensión/feature.

Regresion Lineal

$$\text{RSS}(w) = \sum_i^n (y_i - f(x_i))^2 = \sum_i^n \left(y_i - w_0 - \sum_j^d x_{ij} w_j \right)^2$$

$$\text{RSS}(w) = (\mathbf{y} - \mathbf{X}w)^T (\mathbf{y} - \mathbf{X}w) \quad \text{En forma matricial}$$

Vamos a buscar los parámetros $\mathbf{w}$ que minimicen RSS por medio del método de Cuadrados Mínimos (Least Squares).

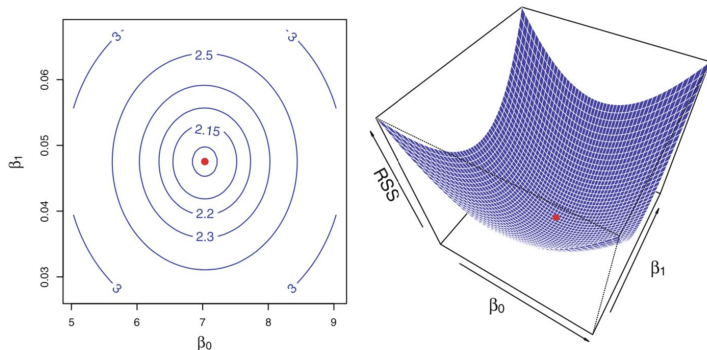
$$\min_w \|\mathbf{X}w - \mathbf{y}\|^2$$

Regresión Lineal: Least Squares

Vamos a buscar los parámetros $\mathbf{w}$ que minimicen RSS por medio del método de Cuadrados Mínimos (Least Squares).

$$\min_w \|\mathbf{X}w - \mathbf{y}\|^2$$

El problema de encontrar w es un problema de optimización convexo! Es decir que hay un mínimo global. Derivando la función de RSS para el caso de regresión lineal podemos obtener una única solución de los valores de los parámetros $\mathbf{w}$ del modelo (ojo! no son hiper-parámetros).



$$\frac{\partial \text{RSS}}{\partial w} = 0$$

$$\hat{w} = (\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}^T\mathbf{y}$$

Ejercicio

Suponer 3 modelos distintos (a,b,c) ya entrenados y caracterizados con sus respectivos parámetros $\mathbf{w}$.
Dados los datos de evaluación (X ,Y) calcular el MSE y R2 de cada modelo y seleccionar el mejor.

$$y = w_0 + w_1 x \quad \begin{array}{lll} w_0^{(a)} = 70 & w_0^{(b)} = 50 & w_0^{(c)} = 60 \\ w_1^{(a)} = -1.5 & w_1^{(b)} = 1 & w_1^{(c)} = -0.5 \end{array}$$

$$X = [30, 28, 32, 25, 25, 25, 22, 24, 35, 40]$$

$$Y = [25, 30, 27, 40, 42, 40, 50, 45, 30, 25]$$

Regresion lineal en python

```
import numpy as np
from sklearn.linear_model import LinearRegression

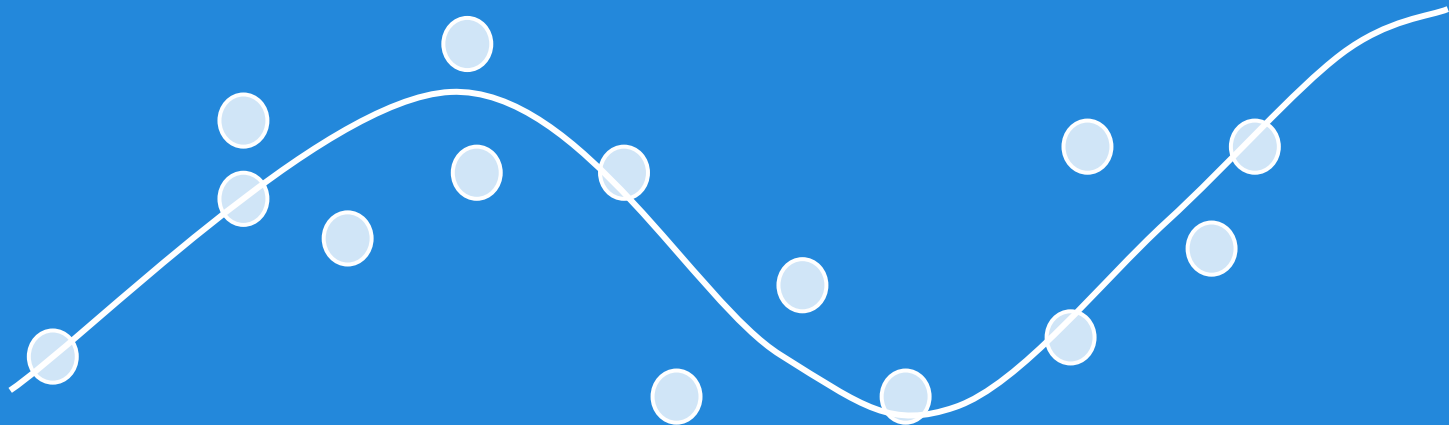
# dataset de entrenamiento
X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
#  $y = 1 * x_0 + 2 * x_1 + 3$ 
y = np.dot(X, np.array([1, 2])) + 3

# ajuste modelo de regresion lineal
reg = LinearRegression().fit(X, y)

# evaluo el modelo en los datos de entrenamiento
reg.score(X, y)
1.0

# me fijo los coeficientes aprendidos durante el entrenamiento
reg.coef_
array([1., 2.])

# uso el modelo entrenado en datos nuevos de test
reg.predict(np.array([[3, 5]]))
array([16.])
```



Regresión: Pipeline de entrenamiento

Pipeline: Train, Validate, Test Model

**Dividir
Train y
Test**

**Cross Validation &
Hyperparameter
search con Train Set
(utilizando Xtrain e
Ytrain)**

**Selección del
mejor modelo**

**Predecir Y para
muestras de Test
(Xtest) sin mostrarle
al modelo las Ytest
(verdaderas labels).**

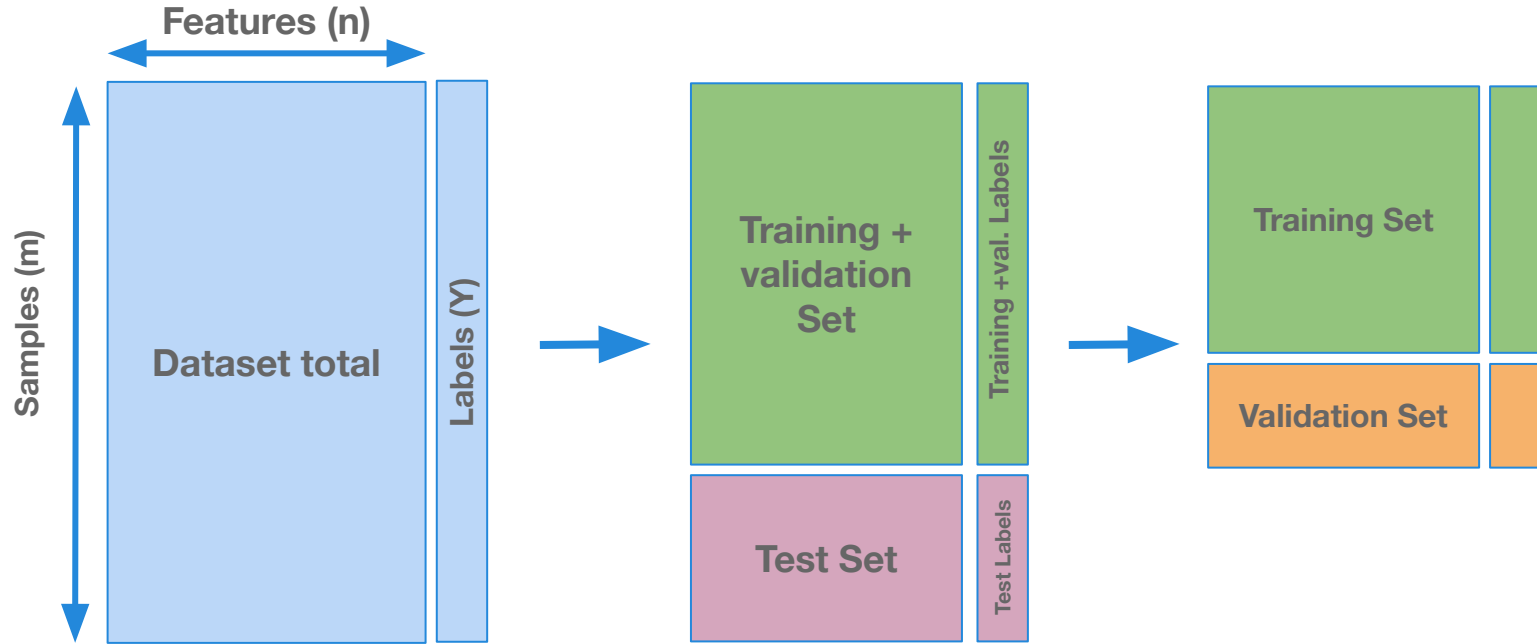
**Evaluar
resultados de
regresión en
test (comparar
Ypred vs Ytest)**

Hiper-parámetros

$f_{w,\lambda}(x)$ funcion de decision
 w parametros
 λ hiper-parametros

Los modelos están caracterizados por parámetros que son aprendidos durante el entrenamiento al ser expuestos a los datos. Adicionalmente los clasificadores tienen hiper-parámetros que definen la familia de funciones que se pueden aprender. Por ejemplo, un hiper-parámetro podría ser el grado de una función polinomial. Los hiper-parámetros no son aprendidos por un algoritmo, son prefijados por el usuario. Los hiperparametros son útiles para poder determinar la complejidad y flexibilidad del clasificador. Por medio de una técnica llamada validación cruzada (cross validation) determinaremos cual la configuración del hiper parámetro que minimiza el error de clasificación.

Train, Validation, Test sets.



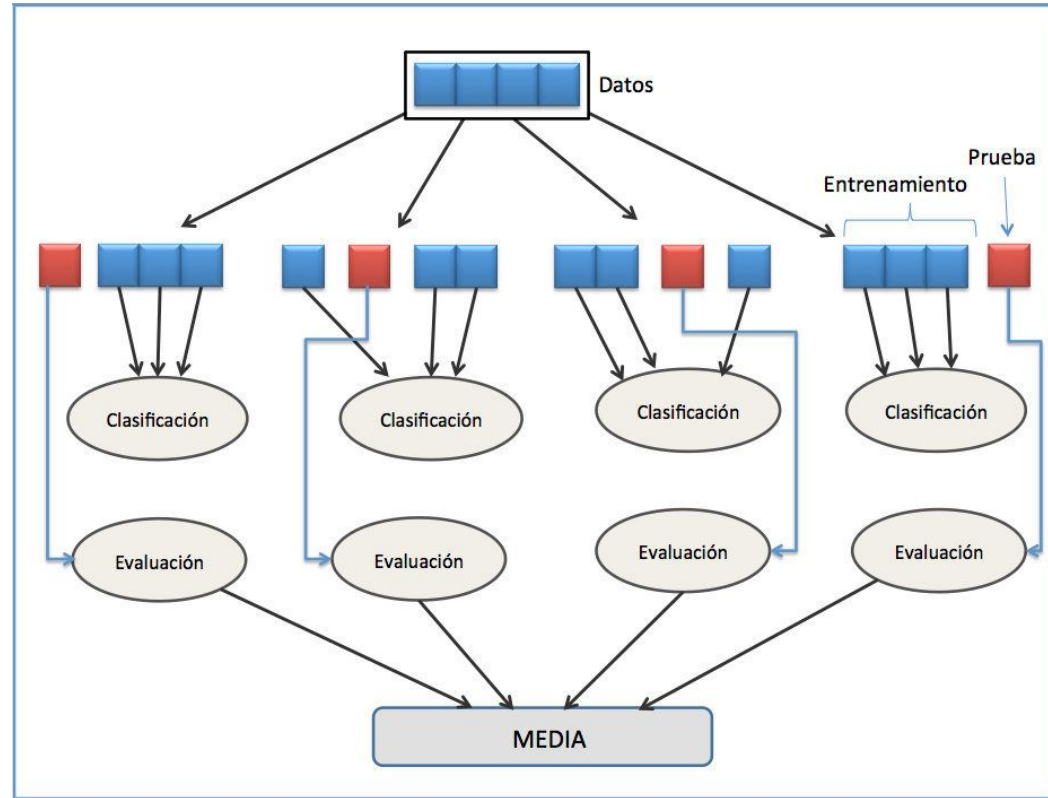
La función de regresión aprenderá utilizando el train set (samples + labels). Luego estimará un valor Y_{pred} para las muestras de test (sin mirar las etiquetas de test). Finalmente se medirá el error de regresión en test.

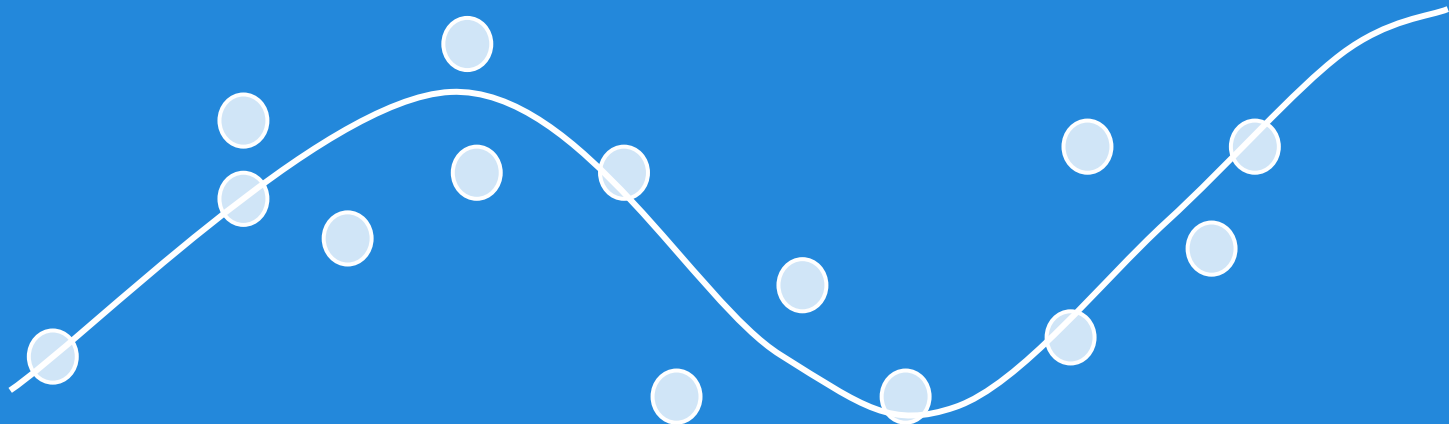
Cross - Validation en training set

“**Cross validation**” (CV) se realiza con las muestras de entrenamiento. Consiste en dividir nuestro training set en **K folds** (K porciones) e iterar K veces.

En cada iteración, una porción se utiliza como validación independiente y el resto como train. En cada iteración se entrena un modelo con train y se evaluará el resultado de regresión con validación. Luego se realizará un promedio de la exactitud de regresión de las k iteraciones.

Cross validation sirve para poder estimar el error estadísticamente. Además si existen varios hiperparametros a cada uno se estima su error por cross validation y se preserva el hiper parámetro que menor error promedio de cross validation genere.





Regresión: Ridge Regression y regularizacion

Regularizacion: Ridge Regression

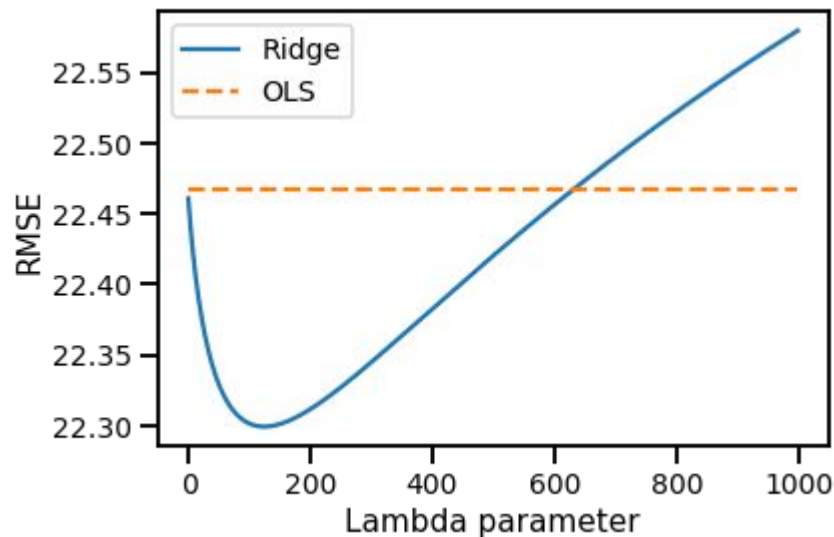
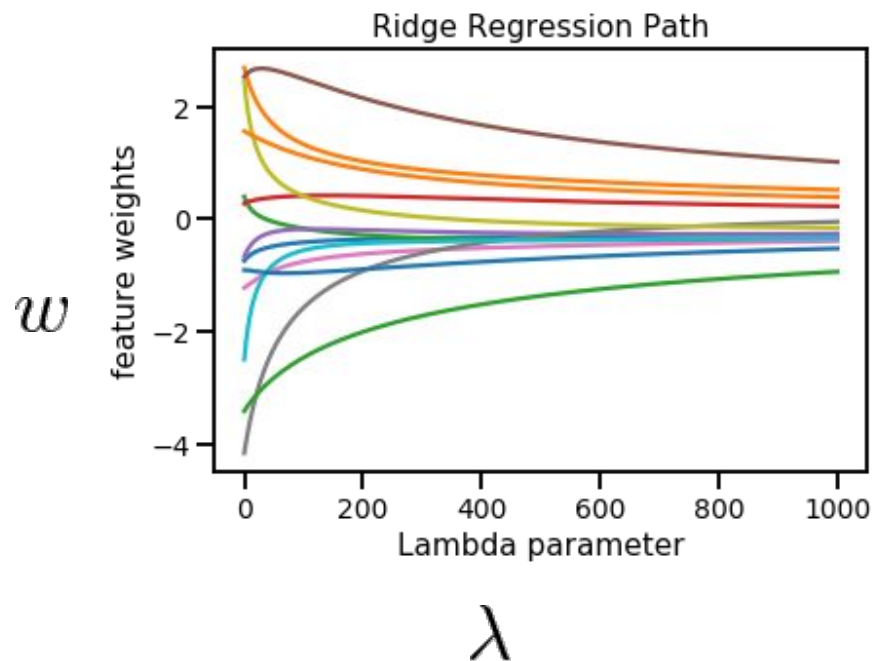
$$\hat{w}^{\text{ridge}} = \underset{w}{\operatorname{argmin}} \left\{ \sum_{i=1}^N \left(y_i - w_0 - \sum_{j=1}^d x_{ij} w_j \right)^2 + \lambda \sum_{j=1}^d w_j^2 \right\}$$

La regresión Ridge impone una penalización a los parámetros $\mathbf{w}$ haciendo que estos tiendan a cero en caso de que no sean tan importantes. Se limitará la norma L2 del vector $\mathbf{w}$

$$\|\mathbf{w}\|_2 = (|w_1|^2 + |w_2|^2 + \dots + |w_d|^2)^{1/2}$$

De manera tal que $\|w\|_2 < t$

Regularizacion: Ridge Regression



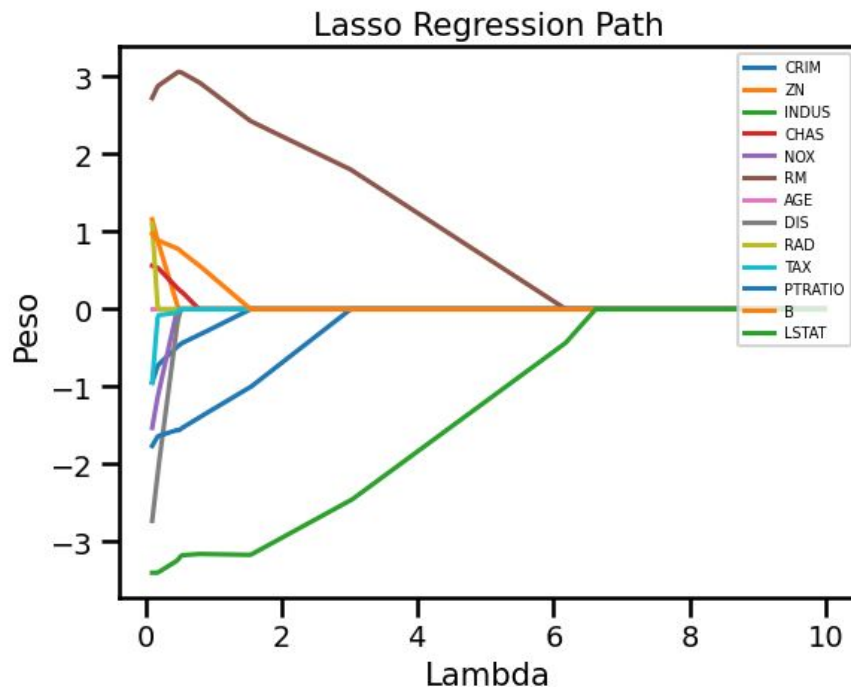
Cuanto mayor sea la penalización entonces más parámetros del vector $\mathbf{w}$ se aproximan a cero. Lambda es un hiperparámetro y se obtiene por validación cruzada. Existe una configuración de lambda que minimiza el error

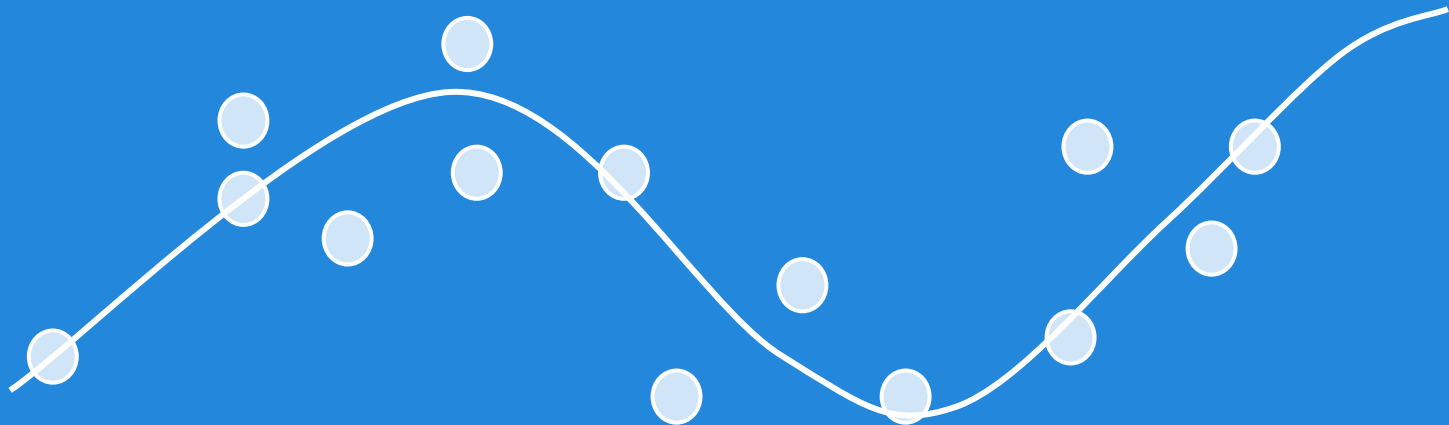
Regularizacion: Lasso Regression

Norma L1 : Lasso

$$\|\mathbf{w}\|_1 = |w_1| + |w_2| + \dots + |w_d| \quad \|\mathbf{w}\|_1 < C$$

Si le imponemos una restricción/penalización a la norma L1 de un vector lo que sucederá es que muchos valores de w tomarán el valor $w_i = 0$ para poder satisfacer la restricción y solo unos pocos serán distinto de cero.

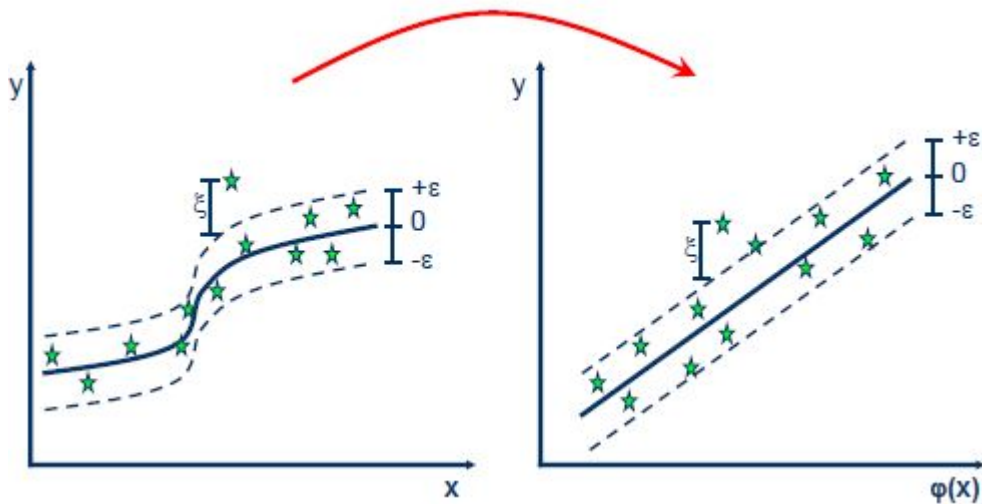




Regresión: Support Vector regression

SVR: Support Vector Regression

- Construye una función Lineal (hiperplano)
- Determina un margen/radio (epsilon) como función de costo y trata de que todas las muestras estén dentro del margen (o “tubo”).
- Con kernels se puede mapear no linealmente mis muestras a otro espacio donde el hiperplano lineal funcione.

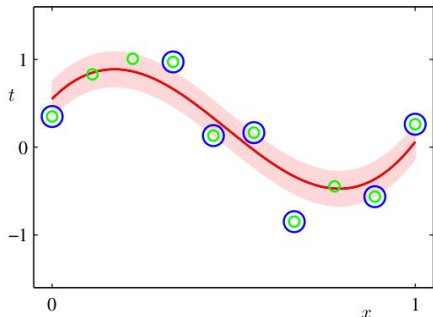


$$\min \frac{1}{2} \|w\|^2 \quad \begin{aligned} y - wx_i - b &< \epsilon \\ -y + wx_i + b &< \epsilon \end{aligned}$$

SVR: Support Vector Regression

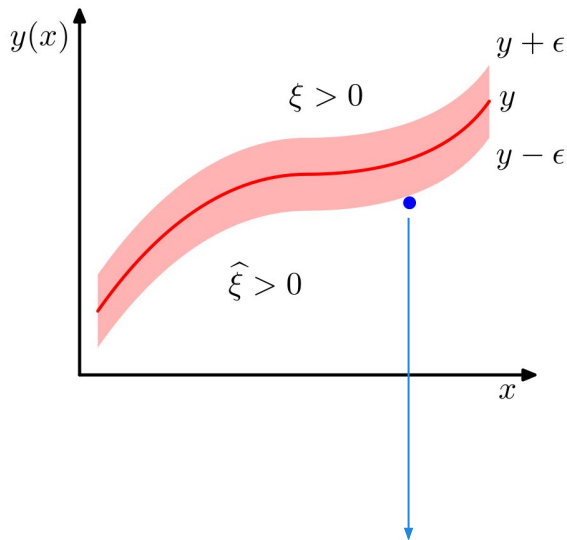
Las muestras que caigan fuera del “tubo” tendrán “ $\xi > 0$ ” y las que estén dentro del tubo “ $\xi = 0$ ”. El hiper-parámetro es una función **C** que penaliza muestras fuera del “tubo”.

$$C \sum_{n=1}^N \xi_n + 1/2 \|w\|^2$$



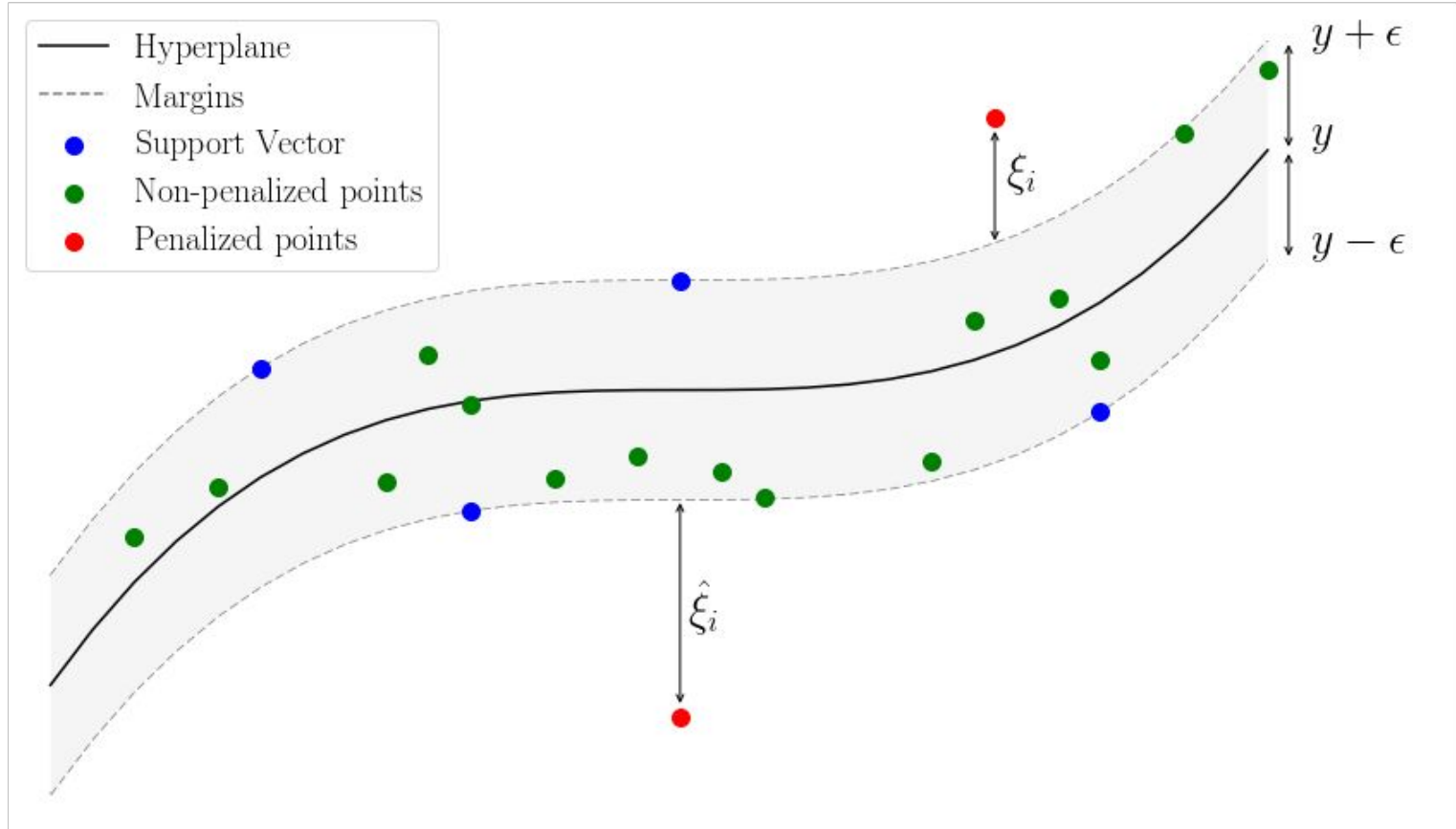
Si el error entre predicción (Y_{pred}) y etiqueta verdadera (Y_{real}) es menor que epsilon entonces se determina que el error es 0.

Solo algunas muestras definirán el “tubo” de predicción y serán llamadas “support vectors”.

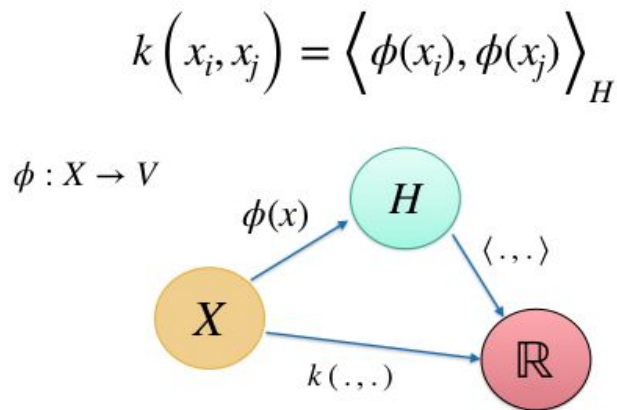


Podemos “permitir/flexibilizar muestras fuera del “tubo” de predicción del modelo.

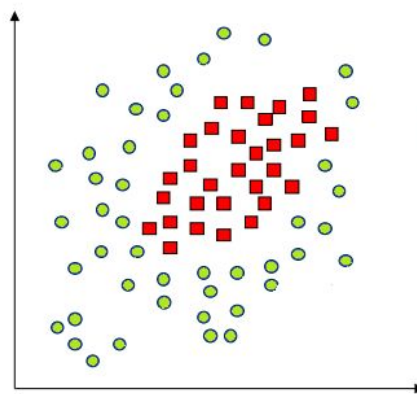
SVR: Support Vector Regression



Kernels in regression: SVR “Kernel Trick”

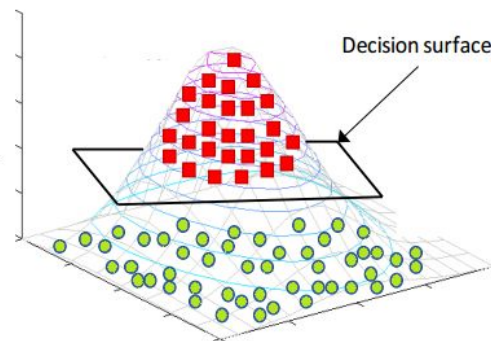


No separables linealmente



kernel

Separables linealmente por un hiperplano.



Los kernels son funciones de similitud entre muestras. Mapean nuestros datos a una dimensión desconocida (de Hilbert) donde son linealmente separables. Allí en ese nuevo espacio donde son mapeadas las muestras se aplican los productos internos (o similitud). Cuando usamos SVR, podemos aplicar un kernel para facilitar la regresión, es decir que el hiperplano estará afectado por el kernel de manera **no lineal**.

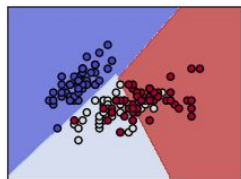
SVR: Hiper Parametros

Kernels más frecuentes: Gaussian, Linar, Polynomial

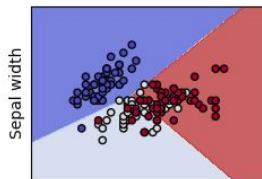
$$K_{gaussian}(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

$$K_{lin}(x_i, x_j) = \langle x_i, x_j \rangle$$

$$K_{poly}(x_i, x_j) = (\langle x_i, x_j \rangle + R)^d$$

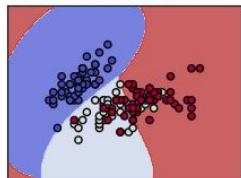


Sepal length

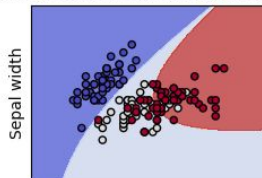


Sepal length

SVC with RBF kernel



SVC with polynomial (degree 3) kernel



Sepal width

Cada kernel hará que el SVR genere distintos tipos de regla de regresión. Los **kernels gaussianos y polinomiales generarán funciones no lineales más complejas** y el lineal o el polinomial de bajo grado más sencillas. ¿cuando usamos un kernel complejo y cuando uno sencillo?

A CODEAR



JAKE-CLARK.TUMBLR

